

最適化の測り方 — 静的と動的は別物

今日のゴール

最適化の効果を測る物差しを 2 つ作る。**静的命令数**（出力に命令が何個あるか）と**動的命令数**（実行時に何回実行されたか）である。この 2 つが桁違いに食い違うことを、自分のコンパイラの出力で確かめる。

1 この回の位置づけ

- 最適化発展シリーズの第 1 回。前提はコマ 16（完成した `final/mycc.py`）
- 以降のすべての最適化の回が、この回で作った物差しを使う。最初に作る理由がこれである
- `mycc.py` は書き換ええない。編集するのは `measure.py` だけ
- ベンチマーク集 `bench/` も、以降の回で共通利用する

2 なぜ測るのか

B1（定数量み込みとピープホール）では「命令数が 16% 減った」と報告した。しかし、その 16% は本当に速くなったのだろうか。

```
int main() {  
    int i; int s;  
    s = 0;  
    for (i = 0; i < 200; i = i + 1) { s = s + i; }  
    return s % 256;  
}
```

このプログラムが出力するアセンブリは 70 命令ほどである。しかし、実行される命令は **7,840** 個ある。ループ本体が 200 回繰り返されるからである。

つまり、ループの外にある命令を 1 個消しても、ほとんど速くならない。逆に、ループの中の 1 命令を消すと 200 命令分速くなる。「命令数が減った」だけでは、どちらの話をしているのか分からない。

3 2つの物差し

	静的命令数	動的命令数
何を数えるか	アセンブリに書かれた命令の個数	実行時に実行された命令の回数
測り方	テキストを数える	qemu で実行を追う
反映されるもの	コードサイズ	実行時間
ループの影響	受けない	大きく受ける

どちらも意味がある。組み込み機器ではコードサイズ（静的）が制約になるし、実行速度が問題なら動的を見る必要がある。

4 動的命令数の測り方

qemu には、実行した命令を記録する仕組みがある。

```
qemu-riscv64 -one-insn-per-tb -d exec,nochain ./prog
```

- `-d exec` は、実行のたびに `Trace ...` という行を出す
- 既定では複数命令をまとめた単位（トランスレーションブロック）ごとに出るので、`-one-insn-per-tb` で **1 命令 = 1 行**にそろえる

出てくる行はこの形をしている。

```
Trace 0: 0x7fe594000100 [003411ef/000000000001038c/0202c078/00000201] _start
      ↑ 実行した命令のアドレス (PC)
```

5 落とし穴: libc が混ざる

そのまま数えると、`loop_sum` は **144,220 命令**になる。自分のプログラムは 7,840 命令なので、**95% が libc の起動処理**である。

```
_start(libc) → 環境変数の準備 → ロケール初期化 → ... → main → ...
```

これでは自分の最適化の効果が埋もれてしまう。そこで、自作した関数のアドレス範囲だけを数える。

1. アセンブリの `.globl` から自作関数の名前を集める
2. `nm` で実行ファイルの各シンボルのアドレスを引く
3. `Trace` 行の PC がその範囲に入っているものだけ数える

これで 144,220 → 7,840 に絞り込める。

補足

なぜログをファイルに書かないのか。-D file でログをファイルに落とすこともできるが、今回の例でも 13MB になる。数百万命令のベンチマークでは数 GB に達する。スケルトンでは `subprocess.Popen` でパイプから 1 行ずつ読みながら数えるようにしている。ディスクを使わず、メモリにも溜めない。

6 ベンチマーク集

`bench/` に 5 本用意した。以降の回でも同じものを使う。

ベンチマーク	特徴
<code>loop_sum.c</code>	単純なループ
<code>matmul.c</code>	3 重ループ、配列アクセスが多い
<code>fib_rec.c</code>	再帰、関数呼び出しが多い
<code>bubble.c</code>	配列の入れ替え、分岐が多い
<code>strops.c</code>	文字列走査、 <code>char</code> アクセス

いずれも `.ans` を持ち、`test_runner.py` で正しさを確認できる。最適化した結果が正しいことの確認にも、そのまま使う。

7 実装

Step	実装対象	内容
1	<code>count_static</code>	アセンブリの行を数える (ラベル・ディレクティブ・コメントは除く)
2	<code>count_dynamic</code> の数え上げ	<code>Trace</code> 行を読み、PC で絞り込んで数える

`nm` の呼び出し (`our_symbol_ranges`) と範囲判定 (`in_ranges`)、`qemu` の起動はスケルトンで用意してある。

```
python3 golden.py          # 5本のベンチマークを両方の物差しで測る
```

8 テスト

```
python3 check.py          # 数え上げの単体テスト (未実装は SKIP)
python3 golden.py         # ベンチマークの正しさ + 静的/動的の表
```

`check.py` の後半は、同じプログラムの反復回数だけを変えて測り、

- 動的命令数はほぼ 2 倍になる
- 静的命令数はほとんど変わらない

ことを確かめる。物差しが正しく機能していることの検証である。

9 測ってみると

教員の実装で `golden.py` を走らせると、こうなる。

ベンチマーク	静的	動的	増幅率
bubble	311	23209	75x
fib_rec	78	73994	949x
loop_sum	70	7840	112x
matmul	386	27321	71x
strops	244	77128	316x

`fib_rec` は静的にはいちばん小さい（78 命令）のに、動的にはいちばん大きい（73,994 命令）。再帰が深く展開されるからである。静的命令数だけを見ていたら、ここが重いとは分らない。

10 発展課題

1. **B1 と比べる:** `advanced/B1_fold_peephole/foldcc.py` でコンパイルしたものを同じ物差しで測り、静的 16% 減が動的では何 % 減になるか調べる
2. **関数ごとに測る:** いまは自作コード全体をまとめて数えている。nm の情報を使えば関数ごとに分けられる。どの関数が重い（プロファイル）が分かる
3. **実行時間と比べる:** `time qemu-riscv64 ./prog` の実測時間と動的命令数を比べる。命令数に比例するか。しないとすればなぜか（メモリアクセスの速さの違い）
4. **命令の種類別に数える:** ロード/ストア、分岐、算術に分けて数える。最適化の狙いどころが見えてくる

補足

コラム: 測らない最適化は当てにならない。「この書き方のほうが速い」という直感は、しばしば外れる。現代の CPU は分岐予測・キャッシュ・命令の並列実行を行うので、命令数が減っても速くならないこと、増えても速くなることがある（07 で扱うループ回転は、静的命令数をまったく変えずに動的命令数を減らす例である）。だからこそ、最適化の議論は必ず測った数字から始める。この回で作った物差しは、以降のすべての回で「本当に良くなったのか」を答えるために使う。